

Supplementary Material for “Embedding-aware Polarization Management in Signed Networks”

I. EXPERIMENTAL DETAILS

This supplementary material provides additional experimental details that complement the main paper. We focus on reproducibility-related information, including hyperparameter search ranges, selected mitigation configurations, large-scale processing, and additional structural statistics.

A. Preprocessing and Evaluation Protocol

All datasets are treated as undirected, unweighted signed graphs. For each dataset, we construct a static signed graph snapshot by keeping the latest sign for each undirected node pair. We then extract the largest connected component and split the resulting edge set into training, validation, and test sets with a ratio of 70%/15%/15%.

Signed link prediction is formulated as a three-class classification task over positive edges, negative edges, and non-edges. Validation and test edges are used only for evaluation, while message passing is performed using training edges only. When mitigation modifies the training graph, the validation and test splits are kept unchanged to ensure a fair comparison between the original and augmented graphs.

B. SGCN Hyperparameter Search

Node embeddings are learned using SGCN. For both the base graph and the mitigated graph, model hyperparameters are selected using validation Macro-F1. The same search space is used for base training and mitigation retraining. Since hyperparameters are selected separately for each dataset and random seed, we report the search range rather than enumerating all seed-specific selected configurations.

TABLE I
SGCN HYPERPARAMETER SEARCH RANGE.

Hyperparameter	Search range
Embedding dimension	{32, 64, 128}
Number of layers	{2, 3, 4}
Learning rate	{0.05, 0.01, 0.005, 0.001, 0.0005}
Epochs	400
Early-stopping patience	10
Random seeds	{0, 1, 2, 3, 4}

C. Mitigation Hyperparameter Search

For EPM mitigation, we tune the pair-selection threshold τ , the maximum selection count $d_{\max}$, and the augmentation

strength γ on the validation set. The negative edge scale is fixed to $\eta = 0.1$ in all mitigation experiments.

For the four small and medium-sized datasets, we use the search range shown in Table II. For the large-scale datasets, Slashdot and Epinions, we use a smaller grid due to the larger graph size and the larger number of community pairs.

TABLE II
MITIGATION HYPERPARAMETER SEARCH RANGE.

Group	τ	$d_{\max}$	γ
Small/medium	{0.5, 0.6, 0.7, 0.8, 0.9}	{2, 3}	{0.5, 1.0, 1.5, 2.0}
Large	{0.2, 0.3}	{3, 4}	{0.5, 1.0, 1.5}

TABLE III
SELECTED MITIGATION HYPERPARAMETERS USED FOR THE MAIN MITIGATION RESULTS.

Dataset	τ	$d_{\max}$	γ
BTC-Alpha	0.5	3	1.5
BTC-OTC	0.5	3	2.0
Wiki-Elec	0.8	2	1.0
Wiki-RfA	0.5	3	0.5
Slashdot	0.2	4	1.5
Epinions	0.2	4	1.5

II. RUNTIME AND LARGE-SCALE PROCESSING

A. Solver-based Polarization Computation

The effective-resistance-based polarization score involves quadratic forms of the form $b^T L^\dagger b$, where $L^\dagger$ is the Moore–Penrose pseudoinverse of the graph Laplacian. Explicitly computing $L^\dagger$ is expensive for large graphs. Therefore, all datasets are evaluated using a solver-based implementation that computes the quadratic form by solving sparse linear systems, avoiding explicit pseudoinverse construction.

B. Large-scale PCS Processing

For Slashdot and Epinions, the number of community pairs can be large. To reduce the cost of polarized community-pair scoring, we apply a pruning step before computing pairwise polarization. We first retain communities with at least 1000 nodes. We then compute a centroid-distance score for each remaining community pair in the aligned embedding space and keep the top 50 most distant pairs for subsequent PCS computation.

C. Runtime Results

We measure the runtime of EPM components using seed 0 and report the mean and standard deviation over three repeated runs. The measurement runtime corresponds to computing the polarization score from a trained embedding and the training graph. The mitigation runtime includes KMeans community construction, polarized community-pair scoring, gray-node scoring, and gray-zone edge construction. SGCN training, SGCN retraining after mitigation, and post-mitigation polarization evaluation are excluded. All runtime measurements are conducted on a single CPU machine equipped with an Intel Xeon Silver processor and 256 GB RAM. SGCN embeddings are trained on an NVIDIA RTX 5090 GPU, but SGCN training and retraining times are excluded from the reported runtime.

TABLE IV
RUNTIME OF EPM MEASUREMENT AND MITIGATION COMPONENTS.

Dataset	Measurement (s)	Mitigation (s)
BTC-Alpha	1.80 ± 0.14	14.28 ± 3.90
BTC-OTC	1.71 ± 0.12	17.97 ± 0.80
Wiki-Elec	1.78 ± 0.11	12.62 ± 0.58
Wiki-RfA	1.88 ± 0.11	25.56 ± 1.05
Slashdot	10.59 ± 4.40	653.97 ± 27.15
Epinions	23.76 ± 7.06	1646.62 ± 114.59

III. ADDITIONAL STRUCTURAL STATISTICS

In the main paper, we compare EPM with random edge augmentation under the same edge budget. In addition to positive average clustering, we measure the number of positive common neighbors associated with added edges. Higher values indicate that newly inserted edges are more consistent with the existing local positive structure. We report this statistic for the small and medium-sized datasets where the full random-augmentation structural comparison is computed.

TABLE V
AVERAGE NUMBER OF POSITIVE COMMON NEIGHBORS FOR ADDED EDGES.

Dataset	EPM	Random
BTC-Alpha	0.12156	0.05962
BTC-OTC	0.13186	0.05444
Wiki-Elec	0.28456	0.26200
Wiki-RfA	0.30820	0.19120

Across the datasets where this statistic is computed, EPM consistently inserts edges with more positive common neighbors than random augmentation. This suggests that EPM introduces less arbitrary local connections and better preserves existing positive local structure.